

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

INTRODUÇÃO AO MLFLOW (EXPERIMENT TRACKING)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDENCIAS	19
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS	24

EMENDAS

O QUE VEM POR AÍ?

Imagine tentar lembrar de cabeça quais combinações de hiperparâmetros e dados você testou ao treinar dezenas de modelos em um projeto de Machine Learning. Difícil, não? Projetos de ML envolvem ciclos contínuos de experimentação, nos quais modelos são treinados repetidamente com diferentes conjuntos de dados, algoritmos e parâmetros. Sem uma abordagem estruturada, o histórico dessas execuções se perde – resultados valiosos deixam de ser comparados e lições aprendidas evaporam. Nesta aula, você aprenderá como evitar esse “laboratório bagunçado” ao adotar o MLflow como ferramenta de Experiment Tracking, trazendo método e organização para seus experimentos de Machine Learning.

Vamos aprofundar o emprego do MLflow para rastrear experimentos de forma consistente e reprodutível. Você aprenderá como registrar automaticamente parâmetros, métricas e artefatos de cada execução de modelo, garantindo que nada se perca. Veremos também como o MLflow se integra ao ecossistema de MLOps – conectando-se com Git para versionar código e com DVC para versionar dados – a fim de proporcionar governança e reprodutibilidade completas nas iniciativas em Machine Learning. Ao longo da aula, exemplos práticos e casos reais vão ilustrar o papel do MLflow no dia a dia de um indivíduo engenheiro de ML, preparando você para aplicar essa ferramenta em projetos profissionais.

HANDS ON

A fase prática desta aula foi projetada para simular um fluxo profissional de experimentação em Machine Learning, aproximando você de cenários reais do mercado. Nesta etapa Hands On, você colocará em prática os conceitos aprendidos, utilizando o MLflow para rastrear um experimento completo: desde a preparação do ambiente até o registro de resultados e modelos.

Iniciaremos com a preparação do ambiente, instalando e configurando o MLflow e executando sua interface web local (MLflow UI). Em seguida, você criará experimentos nomeados e iniciará execuções (runs) para treinar um modelo clássico (por exemplo, um classificador RandomForest no conjunto de dados Iris). Durante esses runs, veremos como logar automaticamente parâmetros e métricas – por exemplo, registrar hiperparâmetros como número de árvores e profundidade máxima, e acompanhar métricas de acurácia em treino e validação.

Você também aprenderá a salvar artefatos importantes: o modelo treinado, gráficos de curva de aprendizado ou qualquer arquivo gerado, tudo versionado pelo MLflow. Ao final, exploraremos a interface web do MLflow para comparar execuções lado a lado: filtrando resultados por tags, ordenando por métricas e identificando visualmente qual experimento produziu o melhor modelo.

Código completo disponível no [GitHub](#) (exemplo de projeto MLflow). Certifique-se de ter Python 3.12+ instalado, juntamente com as bibliotecas necessárias (por exemplo, scikit-learn e mlflow). Siga as instruções do repositório para configurar o ambiente e executar os experimentos.

SAIBA MAIS

Em projetos reais de Machine Learning, é comum realizar dezenas ou centenas de experimentos variando dados e modelos, muitas vezes sem documentação adequada. Esse estilo ad hoc de experimentação traz diversos problemas: perda do histórico de modelos e configurações testadas, dificuldade de reproduzir resultados promissores e risco de colocar em produção modelos subótimos por falta de comparação cuidadosa. Equipes podem se deparar com situações em que ninguém sabe dizer exatamente quais parâmetros geraram determinado resultado, ou por que um modelo escolhido performa pior em produção do que um protótipo anterior – simplesmente porque não houve rastreamento organizado das execuções.

Esses desafios não são novos. Sculley et al. (2015) já apontavam que sistemas de ML acumulam “dívida técnica oculta” quando faltam práticas de manutenção, incluindo o registro deficiente de experimentos. De fato, a incapacidade de reproduzir um modelo posteriormente e a falta de transparência no processo decisório podem comprometer seriamente a evolução de um projeto de ML.

Estudos revelam que um indivíduo cientista de dados típico constrói centenas de modelos durante um projeto, mas sem um meio de gerenciar e relembrar esses modelos, as lições obtidas tendem a se perder. Vartak et al. (2016) introduziram o ModelDB, um sistema acadêmico pioneiro de gerenciamento de modelos, justamente para enfrentar essa dor: ele registrava modelos e métricas automaticamente, permitindo consultas como “Qual experimento deu a maior AUC?”. As lições do ModelDB e de ferramentas como o Sacred confirmaram a necessidade de rastrear experimentos de forma sistemática. Em suma, sem experiment tracking, projetos de ML ficam sujeitos a retrabalho, perda de insights e até erros em produção pela dificuldade de auditoria.

O que é o MLflow?

O MLflow é uma plataforma de código aberto projetada para gerenciar todo o ciclo de vida de modelos de Machine Learning, do experimento inicial até a

implantação em produção. Criado em 2018 por pesquisadores da Databricks liderados por Matei Zaharia, o MLflow nasceu justamente para enfrentar os desafios de experimentação, reprodutibilidade e deploy em ML que discutimos. Diferentemente de plataformas internas de big techs (como o FBLearn do Facebook ou o Michelangelo da Uber) que eram fechadas e limitavam as ferramentas suportadas, o MLflow foi concebido com filosofia de interface aberta – funcionar com qualquer biblioteca, algoritmo ou linguagem.

Os principais componentes do MLflow refletem as dores comuns do desenvolvimento de modelos:

MLflow Tracking – uma API e interface para registro de experimentos (experiment tracking propriamente dito). Permite gravar parâmetros, métricas e artefatos de cada execução de treinamento, associando-os a identificadores únicos de run dentro de experimentos organizados. Essencialmente, o Tracking cria um log estruturado do experimento, respondendo perguntas do tipo: “com quais parâmetros aquele modelo foi treinado e que acurácia ele obteve?”.

MLflow Projects – uma forma padronizada de empacotar código de ML. Define ambientes (por exemplo, dependências em um conda.yaml) e comandos de execução mediante um arquivo de configuração (MLproject). Com Projects, qualquer pessoa pode reexecutar um experimento com apenas um comando, garantindo reprodutibilidade do ambiente e do código.

MLflow Models – um formato unificado para exportar modelos treinados, incluindo os dados necessários e metadados. Um modelo salvo pelo MLflow pode ser carregado em diversos flavors (por exemplo, como objeto do scikit-learn ou como função Python genérica), o que facilita o deploy em diferentes plataformas – local, em lote, como serviço REST ou em nuvens como AWS SageMaker. Em outras palavras, o MLflow Models abstrai detalhes de frameworks específicos e torna o modelo portátil.

Model Registry – componente introduzido posteriormente (MLflow 1.x) que atua como um registro central de modelos versionados. Ele permite organizar modelos por versões e estágios (por exemplo, Staging vs Production), com anotações e aprovações. O Model Registry traz governança ao ciclo de vida: você consegue ver

qual run originou um modelo, promover modelos aprovados para produção e arquivar os obsoletos, tudo com histórico completo.

Nesta aula, daremos ênfase ao MLflow Tracking, que é a base para os demais componentes. Segundo Zaharia et al. (2018), o MLflow foi rapidamente adotado pela comunidade – já em seu lançamento, suportava Python, R e Java e integrava-se via REST API. Isso o tornou um dos projetos open source de ML de crescimento mais rápido (em um ano, atingiu >140 contribuidores e 800 mil downloads mensais). Mais importante que números, o MLflow democratizou as boas práticas de experiment tracking, antes restritas a ferramentas internas de grandes empresas. Com ele, equipes de todos os portes podem estruturar seus experimentos de modo que cada resultado tenha rastreabilidade – quem executou, com que código, com quais dados e parâmetros e onde está o modelo gerado.

MLflow tracking em detalhe

Foco central desta seção, o MLflow Tracking é o módulo que instrumenta nossos experimentos para que nada passe despercebido. Na prática, utilizamos a API do Tracking nos nossos scripts ou notebooks de treinamento: inserimos chamadas como `mlflow.start_run()`, `mlflow.log_param()` e `mlflow.log_metric()` em pontos-chave do código. Dessa forma, conforme o treinamento acontece, o MLflow vai armazenando as informações relevantes. O destino desses dados pode ser local (por padrão, um diretório chamado `mlruns/` é criado no sistema de arquivos) ou remoto (um servidor de tracking centralizado). Cada execução de treinamento vira um Run registrado, identificado por um ID único (um hash) e associado a um Experimento nomeado.

Concretamente, ao chamar `mlflow.log_param("learning_rate", 0.01)`, o MLflow salva a chave-valor `learning_rate=0.01` no registro do experimento em andamento. As chamadas `mlflow.log_metric("accuracy", 0.92)` fazem o mesmo para métricas (neste caso, armazenando uma acurácia de 0,92). Podemos logar múltiplas métricas ao longo do tempo durante o treinamento. Por exemplo, o loss: a cada época de uma rede neural, o MLflow acompanhará a série temporal de cada métrica. Já `mlflow.log_artifact("modelo.pkl")` permitiria guardar um arquivo de modelo treinado ou qualquer outro artefato (um gráfico .png, um arquivo de log etc.).

Ao fim de um run, todas essas informações ficam acessíveis via API ou via Interface Web do MLflow. Importante destacar: o MLflow registra automaticamente metainformações úteis, como a versão do código (se o repositório Git estiver configurado, ele salva o commit hash) e tags do ambiente de execução (por exemplo, versão do MLflow, usuário que executou). Isso significa que, ao olhar um experimento dias depois, você consegue saber exatamente qual código gerou aquele resultado – crucial para reprodutibilidade. Aliás, quando integrado com repositórios de código e dados, o MLflow possibilita recriar totalmente o contexto de um experimento anterior, contribuindo para a ciência de dados reprodutível. Symeonidis et al. (2022) ressaltam que ferramentas de tracking armazenam todas as informações necessárias para reproduzir experimentos de ML.

Outro benefício do Tracking é construir uma espécie de leaderboard interno. Como todos os runs ficam salvos, podemos compará-los e ordenar para identificar o melhor modelo. Por exemplo, imagine que você treinou 50 modelos com combinações diferentes de hiperparâmetros; com o MLflow, você pode rapidamente consultar: “qual combinação de (learning_rate, num_trees) deu a maior acurácia de validação?”. Isso se torna trivial via a UI ou com consultas pela API do tracking. Em suma, o MLflow Tracking atua como um caderno de laboratório automatizado, gravando cada experimento de forma riquíssima em detalhes – algo praticamente impossível de se conseguir manualmente ou com planilhas.

Interface web do MLflow

O MLflow inclui uma interface gráfica acessível via navegador, que serve para visualizar e gerenciar experimentos. Ao rodar `mlflow ui`, um servidor local é iniciado e você pode inspecionar todos os runs registrados. A interface web lista os experimentos na barra lateral (cada experimento com seus runs). Ao selecionar um experimento, você vê uma tabela onde cada linha é um run, com colunas para seus parâmetros, métricas e outras tags. Há funcionalidades de filtro e ordenação: por exemplo, é possível filtrar apenas runs com `modelo = "RandomForest"` ou ordenar decrescentemente pela métrica de acurácia para encontrar o topo.

Uma das capacidades mais úteis da UI são as comparações lado a lado. Você pode selecionar vários runs (marcando checkboxes) e clicar em Compare e a interface exibirá gráficos comparativos das métricas desses runs ao longo do tempo. Isso acelera enormemente a análise: imagine visualizar, em um único gráfico, as curvas de erro de treino de três modelos diferentes – fica evidente qual converge melhor ou se algum sofre overfitting.

A UI se inspira em ferramentas como o TensorBoard do TensorFlow, fornecendo visualizações interativas para processamento de experimentos. Além disso, ao clicar em um run específico, você acessa detalhes completos: todos os parâmetros e métricas logados, a lista de artefatos (com opção de download, se estiver em local acessível) e até um campo de anotações onde membros da equipe podem adicionar comentários sobre aquele run (por exemplo, “modelo candidate a produção”).

Em contexto de equipe, a interface web do MLflow atua como um quadro compartilhado de experimentos, permitindo que todos acompanhem o progresso uns dos outros. Por exemplo, um líder técnico pode abrir a UI e rapidamente ver quais ideias foram testadas nos últimos dias e quais resultados obtiveram, sem precisar pedir relatórios manuais. Essa transparência ajuda na tomada de decisão – é mais fácil promover para staging um modelo quando se pode inspecionar todos os detalhes de como ele foi treinado.

Vale a pena assistir às videoaulas para ver uma demonstração visual do MLflow Tracking UI. Nelas, exploramos a interface com experimentos reais, mostrando como comparar runs e extrair insights instantaneamente. Essa experiência complementa a teoria vista aqui, tornando clara a utilidade prática da ferramenta no dia a dia.

MLflow e reprodutibilidade

Um dos grandes ganhos do MLflow é elevar o patamar de reprodutibilidade experimental em Machine Learning. Reprodutibilidade significa que você consegue repetir um experimento – possivelmente em outro lugar ou no futuro – e obter resultados iguais ou muito próximos dos originais. No desenvolvimento de ML, isso é

notoriamente difícil sem disciplina: envolve ter controle sobre versões de código, versão de dados, semente de aleatoriedade, configurações de ambiente etc. O MLflow, embora não resolva todos esses aspectos sozinho, conecta-se às outras ferramentas que tratam deles para formar um ciclo reprodutível.

Por exemplo, ao treinar um modelo, se o projeto estiver versionado em Git, o MLflow Tracking registra o commit do repositório naquele momento. Se os dados estiverem versionados com uma ferramenta como DVC (Data Version Control), você pode salvar a referência do dataset (um hash ou tag do DVC) como uma tag do MLflow no run. Assim, um run passa a conter a “receita completa” do experimento: código na versão X, dados na versão Y, com hiperparâmetro Z resultando na métrica W. Para reproduzir, basta pegar o código no commit X, os dados no estado Y e executar as mesmas etapas (o MLflow Projects ajuda aqui, padronizando o comando e ambiente). Você deve obter o mesmo resultado W (dentro da variação estatística normal, caso haja alguma aleatoriedade controlada). Essa capacidade de reconstruir experimentos é vital não apenas para pesquisa, mas também em aplicações reguladas: imagine precisar explicar para auditores como um certo modelo de crédito foi gerado – com o MLflow, você pode resgatar todo o histórico e até refazer o treinamento passo a passo para comprovação.

Outra faceta é a reutilização de modelos e processos. Em MLOps, é comum que modelos iniciais sirvam de base para outros (transfer learning, por exemplo). Com o MLflow, você sabe exatamente qual run gerou um modelo base; se outra equipe quiser usá-lo, pode carregá-lo facilmente via MLflow Models e até reproduzir seu treinamento para estender com novos dados. Tudo isso aumenta a confiabilidade do pipeline: reduz o “quem fez o quê” incerto e torna o projeto mais colaborativo e confiável.

Arquitetura do MLflow tracking

Por baixo do capô, o MLflow Tracking é construído de forma modular, separando a camada de registro da camada de armazenamento. Essa arquitetura flexível permite que o MLflow seja usado desde em um projeto solo no laptop até em

um ambiente corporativo com múltiplos usuários e muito volume de experimentos. Os principais componentes arquiteturais são:

Client (API do MLflow): é o lado que o usuário interage – bibliotecas Python, R, CLI etc. O client envia comandos para logar params, metrics, etc., seja escrevendo localmente ou enviando via rede para um servidor.

Tracking Server: é um processo opcional que pode rodar separadamente (especialmente em configurações de time). Ele fornece endpoints de REST API que os clients usam para registrar e consultar dados. Quando você executa `mlflow ui` ou inicia um servidor com `mlflow server`, esse processo de backend passa a gerenciar as requisições, podendo aplicar autenticação, controle de acesso etc. Em ambientes multiusuário, é comum rodar o tracking server centralizado, para todos os experimentos ficarem em um só lugar.

Backend Store (Meta-dados): é a base de dados onde ficam salvos os metadados dos experimentos – parâmetros, métricas (valores numéricos), tags, informações de runs e por aí vai. Pode ser simplesmente um diretório no sistema de arquivos (padrão: `.mlruns` com arquivos YAML ou JSON) ou um banco de dados relacional (MySQL, PostgreSQL, SQLite e outros são suportados). Em cenários pequenos, o file store local basta; em cenários maiores, um banco SQL robusto é recomendado por desempenho e confiabilidade. Por exemplo, em um servidor MLflow de produção, configura-se um PostgreSQL como backend store para suportar muitos usuários simultâneos gravando e consultando.

Artifact Store: é o repositório de artefatos (arquivos) produzidos pelos runs. Enquanto os metadados são pequenos e estruturados (cabem em SQL), artefatos podem ser grandes (um modelo com centenas de MB ou um dataset de validação). O Artifact Store cuida de armazenar esses arquivos. Pode ser o próprio sistema de arquivos local (pasta dentro de `mlruns`) ou serviços de armazenamento em nuvem – o MLflow suporta Amazon S3, Azure Blob Storage, Google Cloud Storage, e até HDFS (Hadoop). Essa separação possibilita otimizações independentes: por exemplo, manter os metadados em um banco para acesso rápido e os artefatos em um bucket S3, que lida bem com escala e volume.

Model Registry (opcional na arquitetura tracking): se habilitado, geralmente requer que o Backend Store seja um banco de dados, pois os recursos de Model Registry (versões de modelo, eventos de transição de estágio) são armazenados lá. O registry não é um componente separado executando, mas sim uma funcionalidade servida pelo Tracking Server sobre a mesma infraestrutura de store.

Em ambientes corporativos de larga escala, essa arquitetura se encaixa bem. Por exemplo, uma empresa pode rodar o Tracking Server dentro de um Kubernetes, com autenticação via LDAP para integração com usuários corporativos. Esse server estaria conectado a um Backend Store Postgres gerenciado e a um Artifact Store S3. Assim, quando um cientista de dados executa experimentos de sua máquina, a API MLflow dele aponta para o Tracking Server remoto (configurando `MLFLOW_TRACKING_URI`). Cada log de métrica vai para o Postgres central, cada modelo salvo vai para o bucket S3 central. Todos os colegas podem então acessar pela UI web centralizada e ver os experimentos uns dos outros. Essa centralização facilita controle de acesso e colaboração. O Tracking Server pode aplicar permissões: por exemplo, garantir que só seu time veja determinados experimentos se necessário. Também escala horizontalmente – pode-se rodar múltiplas instâncias do server atrás de um load balancer para aguentar muitas requisições.

A arquitetura do MLflow oferece flexibilidade: local simples quando você está estudando sozinho, e cliente-servidor robusto quando integrado em equipes maiores. Para nós usuários finais, muitos desses detalhes são abstratos – usar file store vs SQL store não muda comandos do MLflow. Mas é importante entender para planejar uma implantação adequada: se você prevê milhares de runs, configure logo um backend SQL; se vai salvar gigabytes de modelos, aponte o artifact store para um serviço escalável (S3/Blob). Essa capacidade de ajuste garante que o MLflow possa acompanhar o crescimento do projeto sem engessar ou precisar migrar de ferramenta.

Componentes avançados do MLflow

Além do tracking básico que vimos, o MLflow oferece módulos especializados que ampliam suas capacidades no ciclo de vida de modelos. São eles: Projects,

Models e Model Registry – já introduzidos conceitualmente, mas valem detalhes adicionais de como podem ser usados em cenários práticos.

MLflow Projects: este componente aborda a reprodutibilidade e portabilidade do código de experimento. Um Project nada mais é que um diretório (ou repositório Git) contendo seu código de ML e um arquivo MLproject definindo como executá-lo. Dentro do MLproject, você pode especificar:

Um ambiente de dependências – por exemplo, apontar para um arquivo conda.yaml listando pacotes e versões, ou até definir uma imagem Docker a ser usada.

Um ou mais entry points (pontos de entrada), cada um com nome, parâmetros esperados e comando de execução. Por exemplo, um entry point “train” que roda `python train.py --data {data_file} --alpha {alpha}` como no exemplo do MLflow docs.

Com isso, qualquer pessoa (ou processo automatizado) pode rodar seu projeto sem se preocupar em detalhar ambiente ou parâmetros – o MLflow faz por você através do comando `mlflow run`. Você pode inclusive rodar direto de um Git remoto: `mlflow run https://github.com/usuario/projeto.git -P alpha=0.5`. O MLflow vai clonar o repo, resolver as dependências (ex.: criar um ambiente Conda com as bibliotecas corretas) e executar o comando definido, já integrando com o Tracking (ou seja, se dentro do script tem log MLflow, ele registra no tracking normal).

Isso padroniza a execução e facilita incorporação em pipelines. Por exemplo, o AWS SageMaker ou o Azure Machine Learning podem chamar `mlflow run` para disparar treinamentos seguidos. E se o projeto tiver múltiplos steps (data prep, train, evaluate), eles podem ser distintos entry points, chamados em sequência. Em suma, Projects fornece consistência: reduz o “funciona na minha máquina” porque encapsula ambiente, e promove automação, já que serviços externos podem executar um projeto MLflow sem ambiguidade.

MLflow Models: o foco aqui é o deploy. Treinar um modelo é só metade do caminho; precisamos servir esse modelo para predições. Cada framework (sklearn, PyTorch, TensorFlow etc.) tem suas particularidades na hora de salvar e carregar modelos. O MLflow Models define um formato unificado: quando você salva um modelo via MLflow (`mlflow.<framework>.log_model`), ele cria uma pasta que contém

não apenas o arquivo do modelo (por exemplo, `model.pkl` do scikit-learn), mas também um arquivo `MLmodel` com meta-informações, incluindo os flavors suportados.

Um flavor é essencialmente um formato de servimento. Por exemplo, um modelo scikit-learn salvo terá pelo menos dois flavors: “`python_function`” (um wrapper que permite carregá-lo como função Python padronizada) e “`sklearn`” (que permite carregá-lo especificamente com `mlflow.sklearn`). Ferramentas de deploy podem escolher qual flavor usar. Se vamos servir via API REST com MLflow, ele usa o `python_function`. Se quisermos carregar em Spark para scoring distribuído, o MLflow oferece integração de carregar o flavor Spark UDF (User-Defined Function), transformando seu modelo scikit-learn em um UDF executável em Spark, sem você escrever conversões complexas.

Além disso, o MLflow fornece comandos para deploy rápido: `mlflow models serve -m models:/<nome>/<versão>` lança um serviço REST local para aquele modelo (utilizando Flask ou gunicorn internamente). Outras opções incluem exportar um Docker image já configurada com o modelo, ou usar plugins para SageMaker e Azure ML para publicar o modelo nesses serviços cloud em poucos passos. Tudo isso reduz drasticamente o esforço de ir de “modelo .pkl em disco” para “endpoint disponível”. A ideia central é separar o modelo de seu ambiente original, tornando-o um artefato reutilizável. Você pode treinar local com scikit-learn mas depois servir em uma aplicação Java usando o flavor `mleap`, por exemplo, se salvar adequadamente.

Conforme mencionado, o MLflow Model Registry é o repositório central para gerenciar modelos em estágio pós-treinamento. Em uma visão integrada, você usaria o Tracking para gerar vários modelos candidatos e então registrar aqueles de interesse no Model Registry para continuar o ciclo de vida. O Registry permite dar um nome global ao modelo (por ex.: “PrevisorDeChurn”) e versioná-lo: Versão 1, 2, 3, etc. Cada versão aponta para um Run específico do MLflow Tracking (ou seja, está ligado a um experimento que o gerou).

Crucialmente, o Registry implementa o conceito de stage: normalmente, None (registro novo), Staging (fase de testes/integrado a ambiente de validação), Production (em uso produtivo) e Archived (obsoleto). A transição entre stages pode ser feita manualmente via UI ou programaticamente via API, e fica registrada com histórico – quem aprovou a passagem para Production, em que momento, e opcionalmente com

comentários (por exemplo, “Aprovado após atingir 95% de acurácia em validação externa”). Tudo isso cria um fluxo similar a DevOps, mas para modelos: você não precisa mais gerenciar modelos como arquivos soltos ou nomes de arquivos (“model_final2.pkl” etc.) e sim dentro de um sistema que impõe versões e controle.

Com o Registry, integrar um modelo na aplicação fica mais fácil também: a aplicação de produção pode ser programada para sempre puxar a última versão do modelo X com stage=Production. Quando a equipe promover uma nova versão para Production, a aplicação detecta (via API MLflow ou webhook) e carrega o novo modelo. Isso possibilita deploy contínuo de modelos com downtime mínimo. Outras funcionalidades incluem alocar aliases (apelidos) para versões específicas (por ex.: marcar uma versão como “champion” em testes A/B).

Em suma, esses componentes avançados fecham o ciclo de vida complementar ao Tracking: Projects garante que o que foi registrado pode ser reexecutado; Models garante que o resultado treinado pode ser portado para deployment; Model Registry garante que os deployments podem ser organizados e governados. Juntos, fazem do MLflow uma plataforma ampla cobrindo experimentação até operações de modelos.

Performance e escalabilidade

Ao adotar MLflow em ambientes de produção, é importante considerar aspectos de desempenho e escala, para que o sistema de tracking não se torne um gargalo. Em termos práticos, muitos usuários relatam que o MLflow escala bem para a maioria dos usos quando configurado corretamente. Casos de uso extremos, como ‘tracking’ de treinamento de Deep Learning com métricas registradas a cada milissegundo, podem exigir certa moderação (por exemplo, logar métricas em intervalos maiores ou apenas médias agregadas por época). Lembre-se de que o objetivo é capturar o essencial – logs excessivos podem sobrecarregar sem acrescentar insight.

Uma boa prática é simular carga antes de adotar no time: se você espera 10 usuários rodando 100 runs por dia cada, simule 1000 runs e veja se a consulta e a UI ainda respondem rápido. Ajustes podem ser feitos previamente. No geral, o MLflow

foi projetado com leveza – o protocolo de log é simples (JSON sobre HTTP) e as operações no backend são bem distribuídas (cadastro de run, inserções de métricas indexadas por run etc.), então com infraestrutura adequada ele não deve ser o gargalo.

Segurança e governança

Em ambientes corporativos, especialmente aqueles sujeitos a regulamentações, é imprescindível abordar os aspectos de segurança e governança do MLflow. Embora por padrão o MLflow seja aberto (sem autenticação) e simples, ele pode ser inserido em arquiteturas seguras e atender a compliance com algumas medidas.

A versão open source do MLflow não traz uma autenticação pronta, mas se o Tracking Server for colocado atrás de um proxy reverso (como NGINX), ou dentro da infra corporativa, pode-se integrar com sistemas de autenticação como LDAP/AD, OAuth, etc. Por exemplo, configurar um NGINX para exigir login via SSO corporativo antes de encaminhar para o MLflow. Alternativamente, usar soluções gerenciadas (como o Databricks MLflow) fornece autenticação e roles nativamente. Com autenticação, pode-se implementar RBAC (Role-Based Access Control) definindo quem pode registrar experimentos, quem pode ver ou editar modelos no Registry etc. Isso assegura que dados sensíveis de modelos treinados (que às vezes incorporam informações de clientes, por exemplo) sejam acessíveis só por quem deve.

O MLflow também registra automaticamente o usuário (se fornecido) em tags do run e no Model Registry guarda o autor das transições de estágio. Para compliance (ex.: em saúde – HIPAA, ou finanças – Basel etc.), essa trilha é útil, mas pode ser complementada. Boas práticas incluem exigir que no ato da promoção de um modelo a Production seja registrado um ticket de mudança ou aprovação formal (isso pode ser colocado nos comentários do Model Registry). Ferramentas externas podem extrair periodicamente um relatório do MLflow: “quais modelos foram promovidos este mês e por quem”. Essa documentação sistemática ajuda a demonstrar conformidade. Além disso, evitar a possibilidade de alterações silenciosas: idealmente, o Tracking Server deve ser configurado de modo que logs não possam ser apagados sem rastros

(por exemplo, usando bancos append-only ou backups regulares para conferir que não houve manipulação pós-hoc de métricas).

Ao trafegar dados potencialmente confidenciais (parâmetros que podem incluir segredos ou modelos proprietários), use sempre conexões seguras. Se hospedar MLflow server, habilite HTTPS/TLS no endpoint (novamente via proxy ou diretamente). Para dados em repouso, se usar banco de dados e buckets gerenciados, habilite criptografia transparente (a maioria dos serviços cloud permite isso com um flag). Assim, mesmo que haja vazamento de backup, os dados de experimentos não ficam legíveis facilmente. Note que segredos (como chaves de API, senhas de banco) não devem ser logados pelo MLflow – isso é responsabilidade do usuário não chamar `log_param` com senhas. Em vez disso, use gestores de segredo adequados (Vault, AWS Secrets Manager) e não registre segredos no tracking.

Governança inclui decidir por quanto tempo manter os registros de experimentos. Modelos em produção talvez precisem ser guardados por anos (ex.: para poder explicar decisões de crédito de anos atrás). Já experimentos exploratórios podem ser descartáveis após certo tempo. Defina políticas: por exemplo, “todos os runs associados a modelos que não foram aprovados serão apagados após 6 meses”. O MLflow não faz isso automaticamente, mas scripts periódicos podem ser implementados usando a API de delete (`mlflow.delete_run` etc.). Uma abordagem conservadora é nunca deletar runs, apenas arquivar artefatos volumosos (por ex.: deletar artefatos de runs antigos, mas manter métricas/params para histórico leve). Em compliance, se dados de clientes foram usados, pode haver obrigação de exclusão após X tempo – então também planeje purga de runs relacionados. O importante é que haja clareza e documentação dessas políticas para cumprir requisitos legais e gerenciar custos.

Em essência, com algumas camadas extras, o MLflow pode ser executado dentro de padrões corporativos de segurança. Empresas grandes já o utilizam nessa forma – implementações comerciais e open source integradas mostram que é viável ter MLflow em ambientes restritos. Para nós, autores de experimentos, seguir boas práticas (não logar o que não deve, ou usar naming claro etc.) também contribui para uma governança eficaz.

Introdução ao MLflow (Experiment Tracking)

Em particular, destacamos que o MLflow não substitui uma gestão de modelos corporativa completa, mas fornece os blocos principais sobre os quais se constrói governança. Por exemplo, definindo convenções de nomenclatura de experimentos (incluir projeto, finalidade, responsável) ajuda na auditabilidade. Com o tempo, o MLflow se torna o “repositório de conhecimento” sobre a evolução dos modelos da empresa – uma mina de ouro para aprendizado organizacional, desde que mantida segura e bem estruturada.

EMENDAS

MERCADO, CASES E TENDENCIAS

Cases de sucesso e adoção no mercado

Desde seu lançamento, o MLflow rapidamente se tornou uma ferramenta mainstream para experiment tracking. Empresas de tecnologia de diversos portes adotaram-no para gerenciar centenas de experimentos simultâneos em suas plataformas de ML. Um caso notável é o de uma companhia europeia de energia reportado por Zaharia et al. (2018): essa empresa rastreia e atualiza centenas de modelos de previsão de demanda energética usando o MLflow, obtendo uma redução de 73% nos falsos positivos em detecção de anomalias na rede elétrica graças à comparação sistemática de experimentos. O MLflow padronizou o processo de desenvolvimento deles, substituindo planilhas manuais e scripts isolados por um repositório compartilhado de resultados – com isso, um time enxuto conseguiu gerir modelos para cada usina e cada fábrica de seu portfólio, algo impraticável sem automação.

No setor financeiro, a necessidade de auditabilidade impulsiona a adoção do MLflow. Instituições bancárias utilizam o MLflow para justificar decisões algorítmicas em crédito e investimento, pois ele permite recuperar as condições exatas sob as quais um modelo foi treinado (hiperparâmetros, versão de dados) e assim explicar por que uma determinada decisão foi tomada. Uma grande instituição norte-americana integrou o MLflow ao seu fluxo de validação de modelos para conformidade regulatória, mantendo registros de treinamento de modelos de crédito por até 5 anos, conforme exigido pelos órgãos reguladores. Isso trouxe confiança aos auditores internos e externos de que os modelos de credit scoring eram reproduzíveis e documentados.

Outro exemplo vem da área de saúde: HealthTechs que desenvolvem modelos de diagnóstico embarcaram o MLflow para rastrear experimentos com dados clínicos. Uma startup de IA médica, ao criar um modelo de detecção de tumores em imagens, precisa manter não só a performance elevada, mas também provar conformidade com regulamentações. Usando o MLflow, eles conseguem demonstrar que cada versão do modelo foi treinada com dados anonimizados auditáveis e que melhorias de acurácia

ao longo do tempo vieram de experimentos controlados (por exemplo, “versão 3 do modelo treinada com 20% mais dados de pacientes asiáticos para reduzir viés étnico”). Essa transparência acelera aprovações e parcerias com hospitais.

De forma geral, mais de 2000 organizações já utilizavam o MLflow ativamente apenas um ano após seu lançamento, incluindo empresas como Microsoft, Accenture, Booking.com e Zillow, conforme divulgado em eventos da comunidade. A Databricks (empresa mantenedora) incorporou o MLflow a seu produto, o que levou clientes de peso – Shell, Comcast, Regeneron – a empregarem o MLflow em suas plataformas de dados. Isso mostra uma tendência clara: tracking de experimentos se tornou prática padrão em operações de Machine Learning. Inclusive, fornecedores de plataformas de ML proprietárias começaram a oferecer integrações ou funcionalidades semelhantes, validando o conceito introduzido pelo MLflow.

Publicações recentes e tendências tecnológicas

No front acadêmico e de comunidade, várias publicações vêm abordando a evolução do experiment tracking e MLOps:

Enhancing Quantum Software Development with Experiment Tracking” (Gamage et al., 2025) – um [artigo](#) que explora como métodos de tracking (inspirados no MLflow) estão sendo adaptados para pipelines de algoritmos quânticos, mostrando que os benefícios de reprodutibilidade extrapolam o ML clássico.

Podcast “MLOps Coffee Sessions #168: Experiment Tracking in the Age of LLMs” (MLOps Community, 2023) – um episódio com Piotr Niedźwiedz (CEO do Neptune.ai) discutindo como ferramentas de tracking enfrentam novos desafios com os modelos de linguagem gigantes (LLMs). Aborda questões como rastreamento de prompts e versões de parâmetros de modelos de linguagem, indicando que o experiment tracking está evoluindo para suportar não apenas tensores e pesos, mas também artefatos como instruções de prompt tuning. (Disponível no [YouTube](#): “Experiment Tracking in the Age of LLMs – MLOps Podcast #168”).

Relatório Gartner “Emerging Technologies in AI Governance 2024” – destaca que até 2025, 30% das empresas terão um sistema de registro de experimentos de

ML integrado ao seu pipeline de desenvolvimento, contra menos de 5% em 2020. O MLflow é citado como uma das soluções líderes, impulsionando essa tendência ao tornar a implementação relativamente simples. O relatório também prevê uma convergência: plataformas de AutoML incorporando tracking e plataformas de Data Science incluindo registries de modelo nativamente, tudo para garantir compliance e eficiência.

Tendências de mercado

Observamos algumas direções claras para as quais o uso do MLflow e similares está caminhando:

Integração com AutoML e ferramentas low-code: ferramentas de AutoML (como H2O.ai, DataRobot) têm agregado experiment tracking em suas interfaces, muitas vezes permitindo exportar os experimentos para formatos MLflow. Isso democratiza o acesso: mesmo usuários menos técnicos passam a gerar logs ricos de experimentos que podem ser inspecionados posteriormente. O MLflow tende a se tornar invisível para o usuário final, mas presente nos bastidores.

Ênfase em Governança e Explainability: com regulações de IA emergentes (UE, Brasil e outros debatendo legislação), o MLflow ganhará destaque como parte da stack de governança algorítmica. Espera-se um foco maior em recursos de lineage (linhagem) de dados e modelos – possivelmente o MLflow evolua para guardar ainda mais dados de contexto automaticamente (ex.: captura de dataset snapshots). O Model Registry possivelmente incorporará atributos de riscos e aprovação ética, sinalizando se um modelo passou por validação de fairness, por exemplo.

Escalabilidade e SaaS: já existe uma oferta crescente de MLflow como serviço (por Databricks e outras). No mercado, empresas com menos recursos de DevOps tendem a preferir consumir ferramentas como o MLflow via SaaS, pagando pela conveniência de não gerenciar infra. Isso deve aumentar a adoção em empresas médias que antes evitavam a implementação por não terem equipe para mantê-la. Ao mesmo tempo, veremos aprimoramentos para alta escala: por exemplo, suportar

Introdução ao MLflow (Experiment Tracking)

milhões de runs, integrando com data lakes e lakeshouses para armazenar experimentos muito volumosos.

Em síntese, o MLflow se firmou como referência em experiment tracking, e a partir de seu sucesso, todo um ecossistema de práticas e produtos se desenvolveu. O mercado caminha para tornar as fases de experimentação e operação de modelos cada vez mais conectadas e auditáveis e dominar ferramentas como MLflow já é uma habilidade essencial para indivíduos engenheiros e cientistas de dados modernos.

Leitura recomendada e recursos

Livro “Practical MLOps: Operationalizing Machine Learning Models” – Noah Gift et al., O’Reilly (2021). Este livro traz uma visão prática de MLOps e dedica um capítulo ao uso do MLflow dentro de pipelines automatizados, com dicas de integração CI/CD e casos reais. ISBN: 9781098103019.

Blog “Introducing the MLflow Model Registry” – Databricks Tech Blog (17/10/2019), por Clemens Mewald, Matei Zaharia e Cyrielle Simeone. Este [artigo](#) anuncia o Model Registry e discute seus benefícios com exemplos de uso. Uma ótima leitura para entender as motivações de design por trás dessa funcionalidade e ver capturas da interface de registro de modelos.

Vídeo “From Experiment Tracking to Model Deployment” – Palestra no [PyData 2022](#) (YouTube, 38min). Demonstra, com código ao vivo, um fluxo completo usando MLflow: log de experimentos, comparação de métricas e deployment do melhor modelo usando MLflow Models. Ajuda a consolidar como conectar as partes em um caso de uso end-to-end.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprofundou seus conhecimentos sobre o MLflow com foco no rastreamento de experimentos (Experiment Tracking). Vimos por que manter um histórico estruturado dos experimentos é crucial em projetos de Machine Learning e como o MLflow endereça esse problema de forma elegante, registrando automaticamente parâmetros, métricas e artefatos de cada execução. Exploramos a interface web do MLflow e entendemos como ela permite comparar modelos lado a lado, acelerando insights e decisões. Também discutimos como o MLflow se integra a um pipeline maior de MLOps, conectando-se com controle de versão de código (Git), dados (DVC) e ferramentas de deploy contínuo – tudo isso reforçando a reprodutibilidade e governança dos modelos produzidos.

Em resumo, você aprendeu o que é o MLflow, por que ele importa e como aplicá-lo na prática para elevar a qualidade e confiabilidade do ciclo de desenvolvimento de modelos. Com esse conhecimento, você está apto a incorporar o MLflow em seus projetos, garantindo que nenhum insight se perca e que cada modelo desenvolvido tenha sua história bem documentada. Lembre-se de aproveitar as videoaulas associadas a esta aula para ver esses conceitos em ação e solidificar sua compreensão – os conteúdos estão disponíveis e podem ser revisitados sempre que necessário em sua jornada.

REFERÊNCIAS

DATABRICKS. **Databricks Simplifies Machine Learning Model Management At Scale With MLflow Model Registry**. Press Release, 2019. Disponível em: <https://www.databricks.com/company/newsroom/press-releases/databricks-simplifies-machine-learning-model-management-at-scale-with-mlflow-model-registry>. Acesso em: 20 mar. 2026.

KL ISE, J. et al. **Monitoring and explainability of models in production**. In: ICML 2020 Workshop on Deploying and Monitoring Machine Learning Systems, 2020. Disponível em: <https://arxiv.org/pdf/2007.06299.pdf>. Acesso em: 20 mar. 2026.

MEWALD, C.; ZAHARIA, M.; SIMEONE, C. **Introducing the MLflow Model Registry**. Databricks Blog, 2019. Disponível em: <https://databricks.com/blog/2019/10/17/introducing-the-mlflow-model-registry.html>. Acesso em: 20 mar. 2026.

MLFLOW. **MLflow Documentation – Architecture Overview**. Databricks, 2025. Disponível em: <https://mlflow.org/docs/latest/self-hosting/architecture/overview.html>. Acesso em: 20 mar. 2026.

MLOPS COMMUNITY. **Experiment Tracking in the Age of LLMs – MLOps Coffee Sessions #168**. Podcast (YouTube), 2023. Disponível em: <https://www.youtube.com/watch?v=zbkXoONGDGg>. Acesso em: 20 mar. 2026.

SALAMA, K.; KAZMIERCZAK, J.; SCHUT, D. **Practitioners Guide to MLOps: A framework for continuous delivery and automation of machine learning**. Google Cloud whitepaper, 2021. Disponível em: https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf. Acesso em: 20 mar. 2026.

SCULLEY, D. et al. **Hidden Technical Debt in Machine Learning Systems**. Advances in Neural Information Processing Systems (NeurIPS), 2015. Disponível em: <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>. Acesso em: 20 mar. 2026.

SYMEONIDIS, G. et al. **MLOps – Definitions, Tools and Challenges**. arXiv preprint arXiv:2201.00162, 2022.

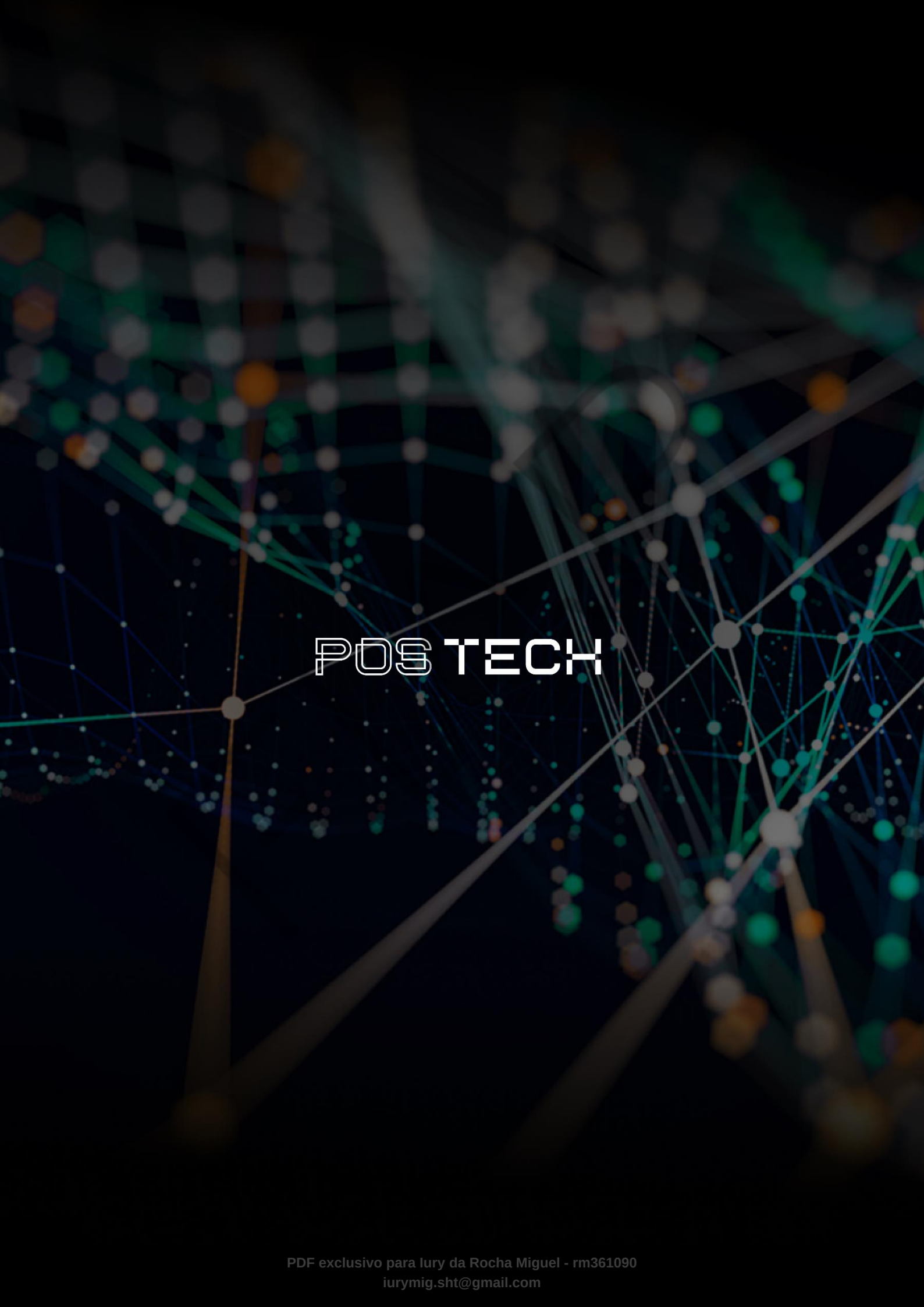
VARTAK, M. et al. **ModelDB: A System for Machine Learning Model Management**. In: Proceedings of the 2016 Workshop on Human-In-the-Loop Data Analytics (HILDA), ACM, 2016. DOI: 10.1145/2939502.2939516.

ZAHARIA, M. et al. **Accelerating the Machine Learning Lifecycle with MLflow**. IEEE Data Engineering Bulletin, v.41, n.4, p.39–45, 2018.

PALAVRAS-CHAVE

MLflow. Rastreamento de Experimentos. MLOps. Reprodutibilidade. Governança de Modelos.

EMENDAS



POSTECH